e_step(x_list, pi, A, phi): the e-step runs the scaled forward-backward algorithm on each sequence separately. First I build the Gaussian emission matrix $B_{n,k} = \mathcal{N}(x_n; \mu_k, \sigma_k)$ in one vectorized `scipy.stats.norm.pdf` call. The forward pass then starts from $\pi_k B_{1,k}$ and repeatedly propagates through the transition matrix and multiplies by the emission:

$$c_n \hat{\alpha}(z_{n,k}) = B_{n,k} \sum_j \hat{\alpha}(z_{n-1,j}) A_{jk}$$

where c_n is the sum of the right-hand side over k. Dividing by c_n at every step keeps $\hat{\alpha}$ a proper distribution, which prevents underflow on the length-1000 sequences. The backward pass starts from $\hat{\beta}(z_n)$ and reuses the same factors, indexed at $n + 1$ rather than n:

$$c_{n+1} \hat{\beta}(z_{n,j}) = \sum_k A_{jk} B_{n+1,k} \hat{\beta}(z_{n+1,k})$$

The posteriors then follow directly:

$$\gamma(z_n) = \hat{\alpha}(z_n) \hat{\beta}(z_n), \quad \xi(z_{n-1,j}, z_{n,k}) = \frac{\hat{\alpha}(z_{n-1,j}) A_{jk} B_{n,k} \hat{\beta}(z_{n,k})}{c_n}$$

In exact arithmetic these already sum to 1. The code renormalizes each γ_n and each $K \times K$ matrix ξ_n anyway, to absorb floating-point error.

m_step(x_list, gamma_list, xi_list): the m-step uses the closed-form maximizers of the expected complete-data log-likelihood. The initial distribution depends only on where each sequence starts, so it is the average of the first γ across the S sequences:

$$\pi_k = \frac{1}{S} \sum_s \gamma_s(z_{1,k})$$

The transition matrix is a ratio of expected counts, with expected $j \to k$ transitions on top and expected departures from j below:

$$A_{jk} = \frac{\sum_s \sum_{n=2}^{N_s} \xi_s(z_{n-1,j}, z_{n,k})}{\sum_s \sum_{n=1}^{N_s - 1} \gamma_s(z_{n,j})}$$

In the code the denominator is `gamma[:-1]`, because the last state has no outgoing transition. It equals the row sum of ξ by the γ/ξ consistency check. For the Gaussian parameters I concatenate all sequences so that every observation counts once. The mean is a γ-weighted average, and the standard deviation is the square root of the γ-weighted variance around the new mean:

$$\mu_k = \frac{\sum_n \gamma(z_{n,k}) x_n}{\sum_n \gamma(z_{n,k})}, \quad \sigma_k = \sqrt{\frac{\sum_n \gamma(z_{n,k}) (x_n - \mu_k)^2}{\sum_n \gamma(z_{n,k})}}$$

fit_hmm(x_list, n_states): runs Baum-Welch by alternating the two steps, starting from the parameters returned by `initialize`, which the first E-step uses. Each iteration maps the current parameters to new ones, $\theta^{(i+1)} = \text{m-step}(\text{e-step}(\theta^{(i)}))$, and the likelihood $p(X|\theta)$ never decreases along the way. Old and new parameters are compared before the old ones are overwritten. The loop stops when the largest absolute change in any of π, A, μ, σ falls below 10^{-4}, or after `max_iters = 100` iterations if it never does.

In testing to find what is an appropriate `max_iters`, I raised it step by step to see where the cap stopped mattering. With a cap of 10, neither dataset reached the tolerance and `seq_long` returned a wrong `A`. With 20, `seq_short` just converged (19 iterations), but `seq_long` stopped at a parameter change of 1.7e-3, still above the tolerance, although its structure already matched the reference. Once the cap exceeds 37, the tolerance fires first on both datasets (19 iterations for `seq_short`, 37 for `seq_long`), so any cap of 50 or more gives identical results and acts only as a safety limit. Choosing 100 as roughly three times the slowest observed run, as a buffer for harder held-out data. On `seq_long`, the parameter change also rose during iterations 3 to 10 before decreasing, so the change between iterations is not monotone, which is another reason not to set the cap tightly.